লজিক অপারেটরের অনুশীলন, ইন্ডেন্টেশন এবং মন্তব্য

পাঠ 10 — 1IOT, SST-ICT-01 — অধিবর্ষ, বৈধ ত্রিভুজ, NOT দিয়ে অ্যালার্ম, ইন্ডেন্টেশন, মন্তব্য; firmware-এর পরিচিতি (Arduino, Harvard ও Von Neumann)

12/10/2026

1 আমরা কোথা থেকে আবার শুরু করি

টীকা

সকালের **পরীক্ষা 1**-এর পরে, এই পাঠে আগের পাঠের **লজিক অপারেটর** ব্যবহার করা হয়: `if`-এর **শর্তের** ভেতরে **AND** (&&), **OR** (||) ও **NOT** (!)। **"V02 - Esercizi Operatori Logici"** পত্রের অনুশীলন থেকে **onlinegdb**-তে পুরো প্রোগ্রাম লেখা হয়। তারপর দুটি **লেখার নিয়ম** (ইন্ডেন্টেশন ও মন্তব্য) এবং শেষ কয়েক মিনিটে firmware নিয়ে প্রথম **আলাপ**।

সংজ্ঞা

বাক্য থেকে অপারেটরে অনুবাদের নিয়ম। অনুশীলনের লেখা পড়ার সময়: **"oppure"** (অথবা) পেলে মানে **OR**, অর্থাৎ দুটি খাড়া দাগ ||; **"e"** (এবং) পেলে মানে **AND**, অর্থাৎ দুটি &&; **"non"** (না) পেলে মানে **NOT**, অর্থাৎ বিস্ময়চিহ্ন !। ইংরেজিতে "e" হলো **and**। **"diverso"** (আলাদা, অসমান) লেখা হয় বিস্ময়চিহ্ন ও তারপর সমান দিয়ে, `!=`, শুধু = দিয়ে নয়।

2 অনুশীলন: অধিবর্ষ

উদাহরণ

প্রশ্ন (পত্রের অনুশীলন 10)। C-তে একটি প্রোগ্রাম লেখো যা একটি বছর চায়। বছরটি 400 দিয়ে বিভাজ্য **অথবা** (4 দিয়ে বিভাজ্য **এবং** 100 দিয়ে **বিভাজ্য নয়**) হলে প্রোগ্রাম ছাপাবে "Anno bisestile" (অধিবর্ষ), নাহলে "Anno non bisestile" (অধিবর্ষ নয়)। **টীকা: বিভাজ্যতার জন্য `%` অপারেটর ব্যবহার করো, যেমন `anno % 4 == 0`।**

ধাপ 1: বছরের ভেরিয়েবল

2026-এর মতো একটি বছর রাখতে কোন **ডেটা টাইপ** লাগে? শিক্ষক ক্লাসের সাথে জোরে ভেবে বিকল্পগুলো বাদ দেন:

- একটি `char` (এই কোর্সে 0–255): **না**, 2026 ধরে না;
- একটি `float`: **না**, বছরে দশমিক থাকে না, অর্থ হয় না;

- `int` হলো **একমাত্র** **যুক্তিসঙ্গত** **পছন্দ**।

তাই `int anno;`। তারপর `printf` দিয়ে ব্যবহারকারীর কাছে মান চাওয়া হয় (“Inserisci l’anno”) এবং `%d` ও `&anno` সহ `scanf` দিয়ে নেওয়া হয়।

ধাপ 2: শর্ত

প্রোগ্রামকে “bisestile” বা “non bisestile” ছাপানোর মধ্যে বাছতে হবে: `if-else` লাগে। শর্ত একটু একটু করে তৈরি হয়।

1. **400** **দিয়ে** **বিভাজ্য**: anno ও 400-এর **ভাগশেষ** (**মডুলো** অপারেটর `%`) **শূন্য** হলে সংখ্যাটি 400 দিয়ে বিভাজ্য, অর্থাৎ `(anno % 400 == 0)`। বন্ধনী “নিরাপত্তার জন্য” দেওয়া হয়।
2. লেখায় আছে “**অথবা**”: **OR** অপারেটর `||` লাগে, তারপর বন্ধনীতে আরেকটি শর্ত।
3. **4** **দিয়ে** **বিভাজ্য**: `(anno % 4 == 0)`।
4. লেখায় আছে “**এবং**”: **AND** অপারেটর `&&` লাগে।
5. **100** **দিয়ে** **বিভাজ্য** **নয়**: সংখ্যা বিভাজ্য না হলে ভাগশেষ **শূন্য** **থেকে** **আলাদা**, অর্থাৎ `(anno % 100 != 0)`।

উদাহরণ

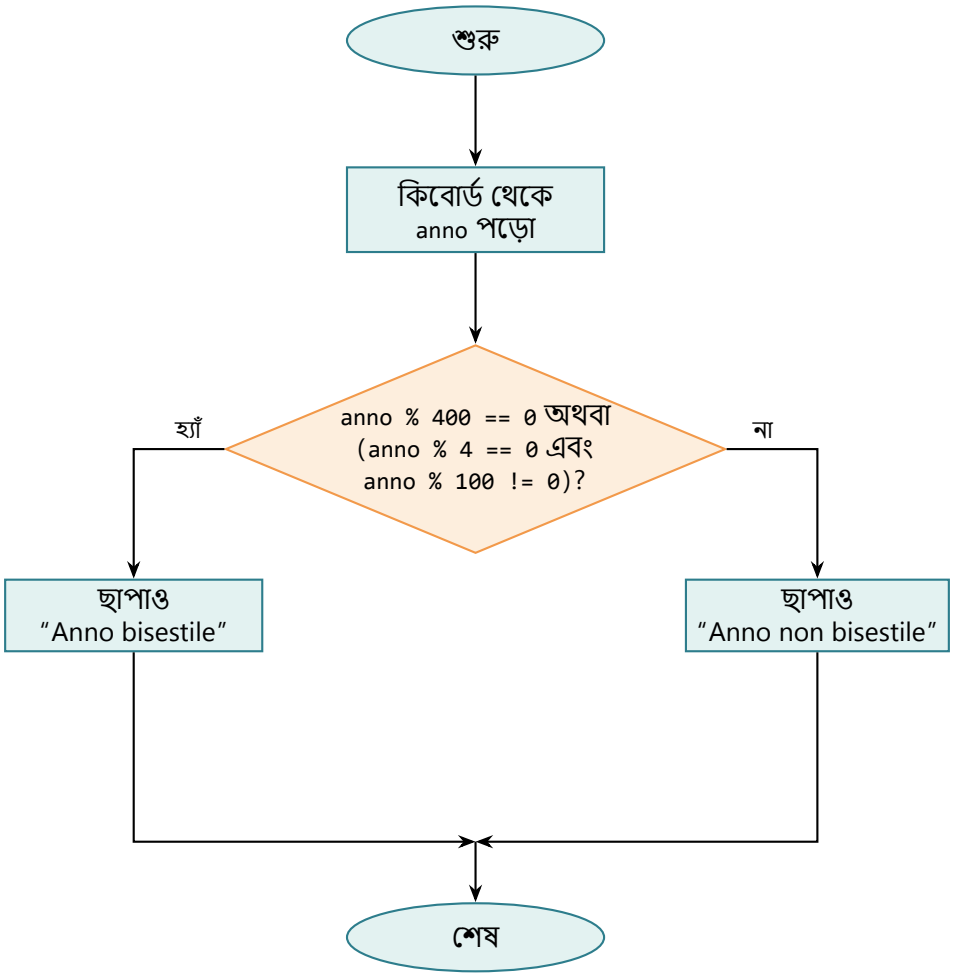
ক্লাসে **লেখা** **কোড** (শুরুর মন্তব্যে অনুশীলনের লেখা, **onlinegdb** থেকে যেমন কপি করা, তাই ইতালীয়তে):

```c
/*
Anno Bisestile: Scrivere un programma in C che richieda un anno.
Se l'anno e' divisibile per 400 OPPURE (divisibile per 4 E NON divisibile per 100),
il programma stampera "Anno bisestile", altrimenti stampera "Anno non bisestile".
(Nota: usa l'operatore % per la divisibilita', es. anno % 4 == 0)
*/
#include <stdio.h>

int main()
{
    int anno;
    printf("Inserisci l'anno: ");
    scanf("%d", &anno);

    if ( (anno % 400 == 0)  ||  ( (anno % 4 == 0) && (anno % 100 != 0) ) )
    {
        printf("Anno bisestile");
    }
    else
    {
        printf("Anno non bisestile");
    }

    return 0;
}
```



ছবি/নকশা 1: অধিবর্ষের ফ্লোচার্ট: একটিই সিদ্ধান্ত, OR ও AND দিয়ে তৈরি একটি শর্ত।

টীকা

প্রোগ্রাম পরীক্ষা করা। ক্লাস বিভিন্ন বছর দিয়ে প্রোগ্রাম পরীক্ষা করেছে: **2000** অধিবর্ষ (400 দিয়ে বিভাজ্য); **2025** ও **2026** অধিবর্ষ **নয়**; 2022-ও নয়। অধিবর্ষে **একদিন বেশি** (ফেব্রুয়ারি 29 দিন)। শিক্ষকের একটি টাইপের ভুল (`% 100`-এর বদলে `% 101`) ক্লাস ধরিয়ে দেয় এবং ঠিক করা হয়: শর্তে লেখা **সংখ্যাগুলো সবসময় যাচাই করো**।

সতর্কতা

“**AND** আগে, **OR** পরে” নিয়মটি ভাগ্যের ওপর ছাড়া যাবে না: প্রতিটি **সরল শর্তের** চারপাশে এবং AND ব্লকের চারপাশে **গোল বন্ধনী** পড়া সহজ করে। পরে অপারেটরের অগ্রাধিকারের অংশ দেখো।

3 একটি নতুন নিয়ম: ইন্ডেন্টেশন

সংজ্ঞা

ইন্ডেন্টেশন (**indentazione**) হলো কোডকে গুছিয়ে ডানে **সরানো**। শিক্ষক বোর্ডে লিখেছেন ("একটি N ভুলে গেছি": **INDENTAZIONE**) এবং আজ থেকে এটি **বাধ্যতামূলক**। নিয়ম:

- বন্ধ **কার্লি বন্ধনী** খোলা বন্ধনীর **ঠিক খাড়া নিচে** থাকে, একই কলামে;
- বন্ধনীর **ভেতরের** সব কোড প্রতি লাইনে একটি **ট্যাব** ডানে লেখা হয়: **TAB** কী চাপো (Q-এর বাঁয়ে);
- ভেতরে আরও বন্ধনী থাকলে তাদের ভেতরের অংশ আরও এক ট্যাব ডানে সরে।

TAB কী কিছু ফাঁক বসায় (সাধারণত তিনটি, ক্লাসের এডিটরে দুই বা চার): **onlinegdb**-র এডিটরে এটি বদলানো যায় না।

উদাহরণ

একই কোড ইন্ডেন্টেশন **ছাড়া** ও **সহ**। প্রথম রূপে বোঝা যায় না একটি বন্ধনী নেই; দ্বিতীয়টিতে সঙ্গে সঙ্গে দেখা যায়।

```
if (a < b)
{
printf("minore");
}
else
{
printf("maggiore");
```

```
if (a < b)
{
    printf("minore");
}
else
{
    printf("maggiore");
```

ইন্ডেন্ট করা কোডে এক নজরে বোঝা যায়, শেষে `else` বন্ধ করার **বন্ধনীটি নেই**: "খুলি, খুলি, বন্ধ করি, আর ওটাও বন্ধ করতে হবে"। সব বাঁয়ে থাকা কোডে ভুলটি অদৃশ্য।

সতর্কতা

কেন বাধ্যতামূলক। ইন্ডেন্টেশন ছাড়াও কোড **কাজ করে** (C-তে "ভাগ্য ভালো"), কিন্তু পড়া যায় না। কোম্পানিতে দলে কাজ হয়: **সহকর্মীদের** কোড পড়ে বুঝতে হয় কী করা হয়েছে। ইন্ডেন্টেশনহীন 15 000 লাইনের ফাইল পড়া যায় না এবং **রক্ষণাবেক্ষণ** ধীর ও সমস্যায় ভরা হয়। আজ থেকে **ইন্ডেন্ট না করা** জমা দেওয়া যেকোনো কোডের মান **শূন্য**: "আমি চাই তা নয়, আমি দাবি করি"।

- সাধারণ ভুল: বন্ধনীর আধারের তুলনায় **অনেক ডানে** সরানো দুই লাইন `printf`। চলে, কিন্তু এটি শৈলীর ভুল।
- **Python**-এর মতো অন্য ভাষায় বন্ধনী নেই: নির্দেশের দল ঠিক হয় **শুধু** ইন্ডেন্টেশন দিয়ে, আর ভুল ইন্ডেন্টেশনে প্রোগ্রাম **কাজ করে না**।

4 অনুশীলন: বৈধ ত্রিভুজ

উদাহরণ

প্রশ্ন (পত্রের অনুশীলন 9)। C-তে একটি প্রোগ্রাম লেখো যা একটি ত্রিভুজের বাহুর (`a`, `b`, `c`) তিনটি পূর্ণসংখ্যা চায়। যদি `a < b + c` **এবং** `b < a + c` **এবং** `c < a + b` হয়, প্রোগ্রাম ছাপাবে "Triangolo valido" (বৈধ ত্রিভুজ), নাহলে "Triangolo non valido" (অবৈধ ত্রিভুজ)।

সংজ্ঞা

পদ্ধতি: শিক্ষক অনুশীলনের **লেখাটি আবার লিখে** কোডে বদলেছেন: "যদি" হয় **`if`**, "এবং" হয় **AND** (`&&`)। তিনটি **পূর্ণসংখ্যার ভেরিয়েবল** ঘোষণা হয়, `printf` ও `scanf` দিয়ে তিনটি বাহু চাওয়া হয় (লেখাগুলো দরকার, "নাহলে কিছুই বোঝা যায় না"), তারপর শর্ত লেখা হয়। ঘোষণা, `printf` ও `scanf` প্রথমে `a`, তারপর `b`, তারপর `c`-এর জন্য।

উদাহরণ

ক্লাসে লেখা কোড:

```c
#include <stdio.h>

int main()
{
    int a;
    int b;
    int c;

    printf("Inserisci il primo lato del triangolo: ");
    scanf("%d", &a);
    printf("Inserisci il secondo lato del triangolo: ");
    scanf("%d", &b);
    printf("Inserisci il terzo lato del triangolo: ");
    scanf("%d", &c);

    if ( ( a < b + c)  && (b < a + c)  && (c < a + b) )
    {
        printf("Triangolo valido");
    }
    else
    {
        printf("Triangolo non valido");
    }

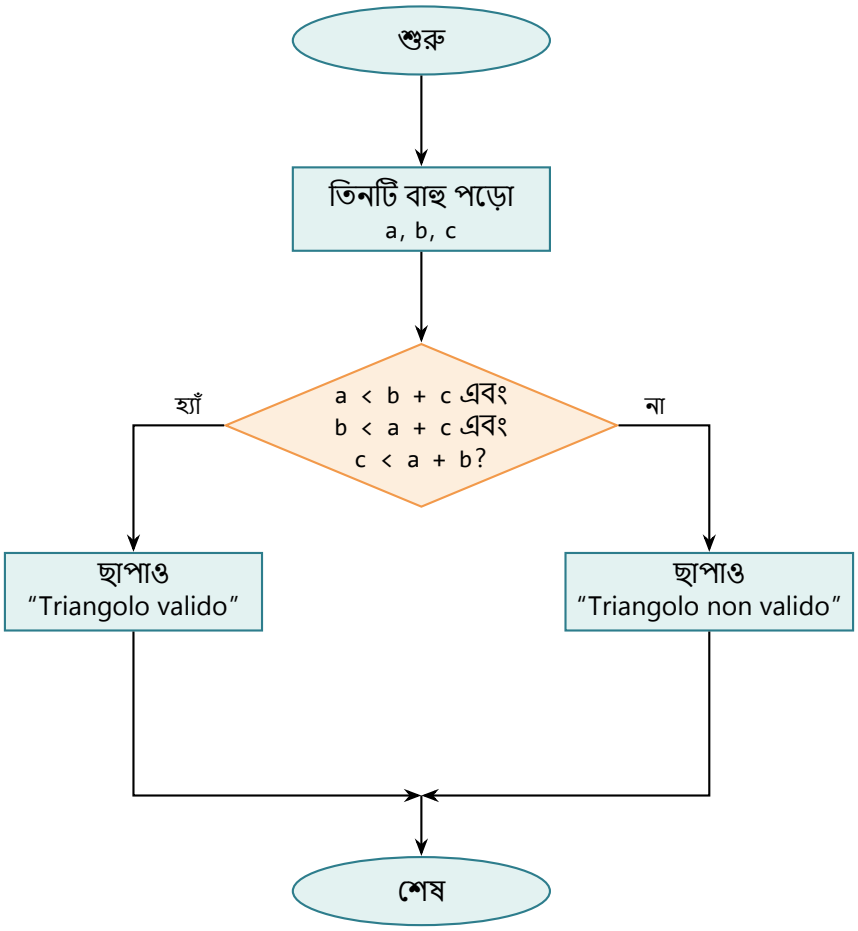
    return 0;
}
```

টীকা

এই শর্ত কেন। ত্রিভুজে প্রতিটি বাহু অন্য দুটির **যোগফলের চেয়ে ছোট** হতে হবে। প্রোগ্রাম তিনটি অসমতা একসাথে যাচাই করে: **একটি** মিথ্যা হলেই **AND** **মিথ্যা** দেয় এবং ত্রিভুজ বৈধ নয়। (একটি বাহু অন্য দুটির যোগফলের **সমান** হলে ত্রিভুজ একটি রেখায় "চ্যাপ্টা" হয়ে যায় এবং কড়া শর্ত ঠিকভাবে তা বাদ দেয়। বাহু ধনাত্মক কিনা যাচাই করতে লেখায় বলা নেই।)

বন্ধনী ও অপারেটরের অগ্রাধিকার

ভেতরের গোল বন্ধনী **ছাড়াও** শর্ত লেখা যায় `(a < b + c && b < a + c && c < a + b)` এবং কাজ করেছে। শিক্ষক: কাজ করে, কিন্তু **বিপজ্জনক**। কম্পাইলার আগে **যোগ**, তারপর **তুলনা**, শেষে লজিক অপারেটর **AND** মূল্যায়ন করে। কেউ ভাবতে পারে কম্পাইলার বদলালে ক্রম বদলায়: "তোমার ভাগ্য ভালো ছিল"। শিক্ষক গোল বন্ধনী **সবসময়** দেন, কারণ তাতে "ভুলের সুযোগ নেই", আর আরও খুঁতখুঁতে হলে যোগের চারপাশেও দিতেন, `(b + c)`। এভাবে কোড সব কম্পাইলারে চলে। "আমরা Arduino ও C-তে কাজ করি, এর বাইরে যাই না।"



ছবি/নকশা 2: বৈধ ত্রিভুজের ফ্লোচার্ট: দুটি AND দিয়ে জোড়া তিনটি তুলনা।

সতর্কতা

সঠিকভাবে বললে: অপারেটরের **মূল্যায়নের ক্রম** (**অগ্রাধিকার**) **C ভাষার মান** দিয়ে স্থির, কম্পাইলার ভেদে বদলায় **না**: আগে গাণিতিক অপারেটর, তারপর তুলনা, তারপর `&&`, তারপর `||`। শিক্ষকের ব্যবহারিক নিয়ম তবু ঠিক: **বন্ধনী** যে কোড পড়ে (ও লেখে) তার সব সন্দেহ দূর করে।

5 অনুশীলন: NOT অপারেটর দিয়ে অ্যালার্ম

উদাহরণ

প্রশ্ন (অনুশীলন পত্র, এডিটরে কপি করা)। Allarme Disattivato: C-তে একটি প্রোগ্রাম লেখো যা ব্যবহারকারীর কাছে অ্যালার্মের অবস্থা চায় (চালু হলে 1, বন্ধ হলে 0)। NOT অপারেটর (`!`) ব্যবহার করে, অ্যালার্ম চালু না হলে প্রোগ্রাম ছাপাবে “Casa sicura” (বাড়ি নিরাপদ); নাহলে “Allarme in corso” (অ্যালার্ম চলছে) (এডিটরে লেখাটি আংশিকই পড়া যায়)।

ভেরিয়েবল ও ইনপুট

1 বা 0 রাখতে একটি **char**-ই যথেষ্ট: `int` লাগে না। শিক্ষক লেখেন `char allarme;`, তারপর একটি `printf` (“Inserisci lo stato dell’allarme (1 attivo, 0 disattivato)”) আর `%d` ও `&allarme` সহ একটি `scanf`। `int`-ও ব্যবহার করা যায়: “ঠিক আছে, কিছু বদলায় না”, কারণ এখনই দরকার অ্যালার্ম চালু না বন্ধ, **সত্য না মিথ্যা**, তা দেখা। (একটি উন্নত ক্ষেত্র হলো keypad, ছোট কিবোর্ড: এতে বোতাম পড়তে হয়, এই বিষয়ে শিক্ষক বছরের শেষে পৌঁছানোর আশা করেন।)

সতর্কতা

কারিগরি টীকা: `%d` একটি **পূর্ণসংখ্যা** (সাধারণত 4 বাইট) পড়ে ভেরিয়েবলে লেখে; ভেরিয়েবলটি **char** (1 বাইট) হলে লেখা ভেরিয়েবল **ছাড়িয়ে** যায় এবং মান অনুযায়ী আচরণ অনির্ধারিত। `char`-এর সঠিক ফরম্যাট `%hhd`। তাই নিচের কোডে ভেরিয়েবলটি **int**, শিক্ষকের অনুমোদিত পছন্দ (“কিছু বদলায় না”)।

NOT দিয়ে শর্ত

অ্যালার্ম চালু **না** হলে ভেরিয়েবলে থাকে **শূন্য**। NOT অপারেটরে: `allarme` 0 হলে `!allarme` হয় **1**, অর্থাৎ সত্য। শিক্ষক শর্ত লেখেন `!allarme != 0`, অর্থাৎ “অ্যালার্মের NOT **শূন্য থেকে আলাদা**”: এতে সমস্যা মেটে। `== 1`-ও লেখা যেত, কিন্তু “NOT ব্যবহার করলে এটি শুধু সত্য হয়: কোথাও লেখা নেই যে এটি 1 হয়, অন্তত সবসময় নয়”। **শূন্যের বড়**-ও ব্যবহার করা যেত, কিন্তু **শূন্য থেকে আলাদা** ভালো: “এটি সহজ, কারণ সংখ্যা ধনাত্মক বা ঋণাত্মক, যা খুশি হতে পারে”।

উদাহরণ

সম্পূর্ণ প্রোগ্রাম (ব্যাখ্যা থেকে পুনর্গঠিত; ক্লাসে লেখা লেখা ফ্রেমে পুরো দেখা যায় না):

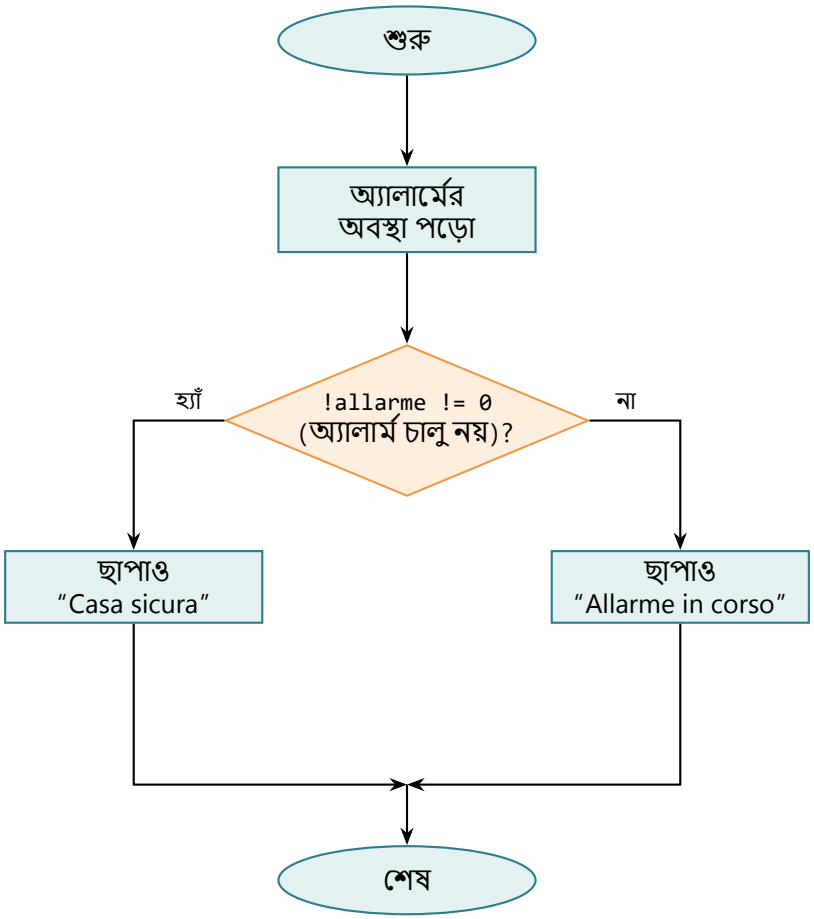
```c
#include <stdio.h>

int main()
{
    int allarme;

    printf("Inserisci lo stato dell'allarme (1 attivo, 0 disattivato): ");
    scanf("%d", &allarme);

    if ( (!allarme) != 0 )
    {
        printf("Casa sicura");
    }
    else
    {
        printf("Allarme in corso");
    }

    return 0;
}
```



ছবি/নকশা 3: অ্যালার্মের ফ্লোচার্ট: শর্তে NOT অপারেটর ব্যবহার হয়।

firmware-এর লেখা: তুলনা ছাড়া `!allarme`

শূন্যের সাথে তুলনা **ছাড়াও** শর্ত লেখা যায়, শুধু `!allarme` রেখে, এবং কাজ করে। কেন? কারণ এটি "যাই হোক উল্টে দেয়" তা নয়: এটি অনেক **firmware** **প্রোগ্রামারের** কৌশল, "আমি নিজেই এভাবে লিখি", অর্থাৎ **কম** **লেখা**। ক্লাসের সাথে শিক্ষক **শূন্য** **থেকে** **আলাদা** লিখেছেন, কারণ আগে ভিত্তি দরকার, উন্নত কৌশল নয়।

সংজ্ঞা

`if`-এর **শর্তকে** উত্তর দিতে হবে **সত্য** **বা** **মিথ্যা**। শর্তের অপারেটর (`a > b`, `x != 0`, ...) সত্য বা মিথ্যা দেয়; কিন্তু `!allarme` নিজেই সত্য বা মিথ্যা দেয়। তাই শূন্যের সাথে তুলনা **লাগে** **না**: `if (!allarme)` যথেষ্ট। নিচের নিয়মটি আগেই দেখা: **শূন্য** **=** **মিথ্যা**, **অন্য** **যেকোনো** **সংখ্যা** **=** **সত্য**।

সতর্কতা

ঝুঁকির উদাহরণ: ভেরিয়েবলে 42 থাকলে 42 **সত্য** আর উল্টে হয় **মিথ্যা**: প্রোগ্রাম সবসময় `else` শাখা চালায়। "ইতিবাচক" বা "নেতিবাচক" বলা হয় না: বলা হয় **সত্য** বা **মিথ্যা**। সংক্ষিপ্ত লেখা ভালোই কাজ করে, কিন্তু "এটি একটু উন্নত: ভুল বোঝার ঝুঁকি অনেক বেশি"। পুরো নিশ্চিত না হলে **সাবধান**: "ভুল খুব কাছেই থাকে"।

একটি নির্দেশে কার্লি বন্ধনী ঐচ্ছিক

`if` বা `else`-এর পরে **একটিই নির্দেশ** থাকলে **বন্ধনী** বাদ দেওয়া যায় এবং প্রোগ্রাম তবু চলে। আমরা **তবু দিই**, কারণ কোড বেশি পাঠযোগ্য হয়: "পারলে এসব এড়িয়ে চলো"।

টীকা

অনুশীলনের মূল্যায়ন। প্রোগ্রামিংয়ের অনুশীলন কীভাবে মূল্যায়ন হবে? "এই প্রশ্নগুলোর উত্তর দাও" ধরনের পরীক্ষায় মানদণ্ড কম; **পরের পরীক্ষা** হবে "**একটি প্রোগ্রাম লেখো** যা এটা, এটা আর ওটা করে"। প্রোগ্রাম হয় কাজ করে নয় করে না, কিন্তু C-তে **ভাগ্যক্রমেও** কাজ করতে পারে (বন্ধনীতে দেখা গেছে)। শিক্ষক প্রোগ্রাম লেখার ক্ষমতাকে গুরুত্ব দিতে চান, তবে ইন্ডেন্টেশন ও মানও গোনা হয়। পরীক্ষা **কম্পিউটারে** হলে **কম্পাইল না হওয়া** প্রোগ্রাম জমা দেওয়া ভারী: কম্পাইলার ভুল দেখায়, আর ঠিক করতে না পারা মানে কীভাবে কাজ করে তা না বোঝা; কাগজে একটি ভুলে যাওয়া কমার ওজন কম। "নম্বর দেওয়া আমাদের প্রাতিষ্ঠানিক দায়িত্ব।"

6 কোডে মন্তব্য

সংজ্ঞা

মন্তব্য (**commento**) হলো কোডে লেখা এমন লেখা যা **কম্পাইলার ধরে না**। এটি কোড পাঠকের বোঝার জন্য। C-তে **দুইভাবে** লেখা যায়:

- **বহু-লাইনের মন্তব্য**: `/*` ও `*/`-এর মধ্যে। দুই প্রতীকের মাঝের সব মন্তব্য, একাধিক লাইনেও;
- **এক লাইনের মন্তব্য**: `//`-এর পরে। শুধু লাইনের শেষ পর্যন্ত: নতুন লাইনে আবার **সাধারণ কোড**।

উদাহরণ

বোর্ডে শিক্ষক প্রতীকগুলো এঁকেছেন: `/*` (খোলা), `*/` (বন্ধ) আর পাশে একটি লাইনসহ `//`। **onlinegdb**-র এডিটরে সব মন্তব্যের লেখার **রং** বদলায় (কমলা): প্রোগ্রামের **বহু-লাইনের মন্তব্য** (`main.c`-এর শিরোনামের মতো) লাইন 1 থেকে শুরু হয়ে লাইন 8-এ `*/` দিয়ে শেষ হয়।

```c
/* এটি একটি মন্তব্য
   একাধিক লাইনে: এখানে যা কিছু
   লিখি কম্পাইলার তা উপেক্ষা করে */

// একটি বার্তা ছাপাই
printf("Ciao");

// ব্যবহারকারীর কাছে একটি সংখ্যা চাই
scanf("%d", &x);
```

শিক্ষক দেখিয়েছেন, `printf`-এর আগে ("একটি বার্তা ছাপাই") বা `scanf`-এর আগে ("ব্যবহারকারীর কাছে চাই...") মন্তব্য দেওয়া যায়: **যা খুশি লেখো**, লাইনের শেষে আবার কোড। আর `/*` দিয়ে খুললে `*/` দিয়ে **বন্ধ না করা পর্যন্ত** সব মন্তব্য হয়ে যায়।

সতর্কতা

কোম্পানিতে মন্তব্য বাধ্যতামূলক। যে সফটওয়্যার লিখে মন্তব্য দেয় না, মালিক তা **ফেরত পাঠায়** ("দেখো, মন্তব্য দাও, নাহলে এগোব না"): কাল যে লিখেছে সে হয়তো সেখানে আর কাজ করবে না, আর কোড **কোম্পানির**, যাকে তা বিক্রি ও রক্ষণাবেক্ষণ চালিয়ে যেতে হবে। পরে যে আসে, মন্তব্য ছাড়া সে জানে না কী লেখা আছে।

7 বিষয় বদল: firmware কী

টীকা

শিক্ষক কোর্সের দ্বিতীয় বিষয় নিয়ে একটি **"20 মিনিটের আলাপ"** শুরু করেন: **firmware**। এখন পর্যন্ত প্রোগ্রামিং বিষয়ে কাজ হয়েছে **কম্পিউটারে**। firmware করা মানে এমন **প্রোগ্রাম লেখা** যা কম্পিউটারে চলে না, বরং **ইলেকট্রনিক বোর্ড** চালায়, যেমন স্কুলের দ্বিতীয় দিনে দেখানো বোর্ড। প্রথম বছরে **Arduino** ব্যবহার হয়; শিক্ষক বছরের শেষের মধ্যে এটি ছেড়ে আরও আকর্ষণীয় কিছুতে যাওয়ার কথা ভাবেন।

উদাহরণ

Arduino কেন শুধু শেখার জন্য। শিক্ষক Arduino পছন্দ করেন না কারণ এটি "সব অতিরিক্ত সহজ করে দেয়": এত সহজ যে কোনো দক্ষতা লাগে না, বাড়ির একটি ব্লেন্ডারের মতো। **পরীক্ষা করতে** এবং নাহলে খুব জটিল ধারণা বুঝতে এটি ভালো, কিন্তু এটি **firmware করা নয়**। বোর্ডটি **আনুষ্ঠানিকভাবে 2005 সালে** আসে (আগে প্রোটোটাইপ ছিল): **USB তার** দিয়ে কম্পিউটারে জোড়া হয়, কী করতে হবে লেখা হয়, আর সে করে।
আগে শিক্ষক বোর্ড ছাড়া শুধু ছোট কালো চিপ, **মাইক্রোকন্ট্রোলার** (বোর্ডের "মস্তিষ্ক"), নিয়ে কাজ করতেন: **ম্যানুয়াল** কিনতে হতো (প্রায় 70–80 হাজার লিরা, আজকের 80 ইউরো), পড়তে হতো, **শুধু** যন্ত্র সেট করতে প্রায় 200 লাইন কোড লিখতে হতো, তারপর তাকে কী করাতে হবে তাতে মন দেওয়া। শুরু করা খুব কঠিন ছিল: Internet কম, YouTube ছিল না, কেউ ব্যাখ্যা করত না। এই বোর্ডে সব সহজ হয়ে গেছে, কিন্তু **ফ্রিজ**, **ওভেন**, **মাইক্রোওয়েভ**, টিভি, মোবাইল খুলে দেখো: পেছনে এমন বোর্ড নেই। **শেখার** স্তরে ভালো, **উৎপাদনের** স্তরে নয়। যে চার বছরে শুধু Arduino শিখবে "তাকে কেউ চাকরি দেবে না"।

সংজ্ঞা

মাইক্রোকন্ট্রোলার (**microcontrollore**) হলো অনেক ধাতব **পা**-সহ একটি **কালো চিপ**। প্রতিটি পা একটি **বৈদ্যুতিক সংযোগ**, অর্থাৎ একটি তার যা যন্ত্র চালু বা বন্ধ করতে পারে: কারেন্ট দিলে **1**, কারেন্ট সরালে **0**। পায়ের চালু/বন্ধ স্তর অনুযায়ী **বাইরে** কিছু ঘটে। চিপের ভেতরে আমাদের লেখা **প্রোগ্রাম** আর প্রক্রিয়া করার **ডেটা** লোড হয়।

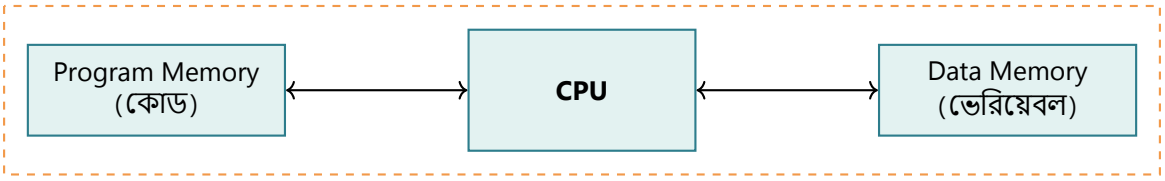
সংজ্ঞা

CPU = **Central Processing Unit**, **কেন্দ্রীয় প্রক্রিয়াকরণ একক**: "মস্তিষ্ক", যা **হিসাব** করে এবং কোন নির্দেশ চালাবে তা বেছে নেয়। চিপে এটি **দুটি মেমোরির** সাথে যুক্ত:

- **Program Memory** (প্রোগ্রাম মেমোরি): প্রোগ্রামারের লেখা **কোড** রাখে;
- **Data Memory** (ডেটা মেমোরি): **ডেটা**, অর্থাৎ সব **ভেরিয়েবল** রাখে। প্রতিবার একটি ভেরিয়েবল তৈরি হলে (`int x;`, `int allarme;`) সেটি ডেটা মেমোরিতে যায়।

onlinegdb-তে কোড লিখে মনে হয় সেখানেই আছে, কিন্তু আসল ভেরিয়েবল তৈরি হয় যন্ত্রের ডেটা মেমোরিতে।

মাইক্রোকন্ট্রোলার (Harvard আর্কিটেকচার)



ছবি/নকশা 4: মাইক্রোকন্ট্রোলারের ব্লক চিত্র: একটি CPU, প্রোগ্রামের একটি মেমোরি আর ডেটার একটি মেমোরি, আলাদা।

টীকা

ক্লাসের প্রশ্ন, করা হয়নি: এমন একটি প্রযুক্তি আছে যা CPU ব্যস্ত না রেখে মেমোরিতে ডেটা সরায়। শিক্ষক: "এখন করার জন্য খুব জটিল", খুব **বিশেষায়িত** বিষয় (প্রথম বছরের নয়); দরকার হলে বিরতিতে বা অন্য সময় কথা হবে।

8 কত মেমোরি? পিসির সাথে তুলনা

উদাহরণ

বাড়ির পিসি। ছাত্রদের কম্পিউটারে: 32 ও 128 গিগাবাইট **RAM** (128 গিগাবাইট = 128-এর পরে নয়টি শূন্য বাইট); 500 গিগাবাইট থেকে 4, 6 ও 12 টেরাবাইট **ডিস্ক**। **Arduino**-তে ডেটা মেমোরি মাত্র **2 কিলোবাইট**। শিক্ষক প্রস্তুতকারকের সাইট থেকে Arduino Uno-র মাইক্রোকন্ট্রোলার, Atmel-এর (এখন Microchip) **ATmega328P**-এর **datasheet** (কারিগরি পত্র) দেখেছেন: প্রোগ্রামের জন্য **32 KB flash** মেমোরি, **1 KB EEPROM**, ডেটার জন্য **2 KB** অভ্যন্তরীণ **SRAM**। "এটা বেশি না কম?" কম্পিউটারের মেমোরি **অসীম গুণ বড়**: "কাজের অনুপাত সম্পূর্ণ আলাদা"।

	Arduino (ATmega328P)	বাড়ির পিসি (ক্লাসের উদাহরণ)
ডেটা মেমোরি (RAM)	2 KB (SRAM)	32–128 GB
প্রোগ্রাম মেমোরি / ডিস্ক	32 KB (flash)	4–12 TB (ডিস্ক)

9 Harvard আর্কিটেকচার এবং Von Neumann আর্কিটেকচার

সংজ্ঞা

প্রোগ্রাম ও **ডেটার** জন্য **দুটি** **আলাদা** **মেমোরি** থাকা যন্ত্রের **Harvard** **আর্কিটেকচার**। এটি একটি নকশার পছন্দ ("একটি নতুন ধরনের কম্পিউটার বানাতে হবে: এভাবে বানাই")।

সংজ্ঞা

CPU আর **একটিমাত্র** **মেমোরি** থাকা যন্ত্র, যাতে **ডেটা** ও **প্রোগ্রাম** দুটোই থাকে, তার **Von Neumann** **আর্কিটেকচার**। উদাহরণ: **পিসি**। ডিস্কে রাখা প্রোগ্রাম চলার জন্য **RAM**-এ লোড হয়: CPU ডেটা আদান-প্রদান করে শুধু RAM-এর সাথে।

উদাহরণ

পিসিতে (onlinegdb) কাজ করার সময় **Von** **Neumann** আর্কিটেকচার; শুধু **Arduino**-তে কাজ করার সময় **Harvard** আর্কিটেকচার। **পরীক্ষার** **প্রশ্ন**: "Harvard ও Von Neumann ধরনের যন্ত্রের পার্থক্য কী?" উত্তর: **Harvard**-এর **দুটি** **মেমোরি**, একটি ডেটার ও একটি প্রোগ্রামের; **Von Neumann**-এর **একটি** **মেমোরি**, যেখানে সব থাকে।

টীকা

কোনটি সেরা? **Harvard** বেশি **দ্রুত**: CPU প্রোগ্রাম মেমোরি ও ডেটা মেমোরির সাথে **একই** **সময়ে** ডেটা আদান-প্রদান করতে পারে (ডেটায় কাজ করতে করতে প্রোগ্রাম লোড করে)। Von Neumann-এ হয় **প্রোগ্রাম** পড়া হয় নয় **ডেটা**: একভাবে বললে "অর্ধেক গতিতে" কাজ হয়।

10 পিসি কেন Harvard নয়: বাজারের পছন্দ

উদাহরণ

তাহলে পিসি Harvard দিয়ে বানানো হয় না কেন? Harvard কম্পিউটারের দাম **অনেক** **বেশি** হতো; "তুমি তখন কিনবে না, আর তুমি না কিনলে আমি বানাব না"। **প্রযুক্তিগত** **পছন্দ** কখনো সেরা হয় না: সেগুলো হয় যা **বিক্রি** **হয়**: কোনো পণ্যের দাম খুব বেশি হলে তা বানানোর অর্থ নেই। একটি "entry level" কম্পিউটারের দাম 7000 ইউরো হলে পুরো **বাজারের** **অংশ** হারিয়ে যেত।

উদাহরণ

প্রসেসরের ক্ষেত্র (গতির স্তরের উদাহরণ)। একই প্রজন্ম ও একই সংখ্যক **core**-এর একটি Intel i7 পাওয়া যায় 3, 4 ও 5 GHz-এ। প্রতিটি গতির জন্য Intel-এর আলাদা উৎপাদন লাইন আছে? **না**: লাইন একটিই এবং **5 GHz**-এর জন্য ভাবা প্রসেসর **তৈরি করে**। কারখানা থেকে বের হওয়ার সময় প্রতিটি চিপ **পরীক্ষা** হয়:

- 5 GHz-এ পৌঁছালে 5 GHz ছাপা হয় এবং বেশি দামে বিক্রি হয়;
- 4 GHz-এ থামলে ("ত্রুটিপূর্ণ, 5-এ যেতে পারে না") 4 GHz ছাপা হয়;
- 3 GHz-এ থামলে 3 GHz ছাপা হয় এবং কম দামে বিক্রি হয়।

শূন্য অপচয়: যা তৈরি হয় সব বিক্রি হয়। ধীরতম রূপ যে কেনে সে কারিগরিভাবে একটি "ত্রুটিপূর্ণ" পণ্য কেনে যা প্রস্তুতকারক জানে: 3 GHz-এ বোঝা যায় না, কিন্তু **overclock** করলে যন্ত্র "আগেই আটকে যায়"। এটি **একই** পরিবারের জন্য: Ryzen 7 ও Ryzen 9 আবার **দুটি আলাদা লাইন** (প্রক্রিয়া একটু আলাদা)। অন্য নামের জন্য (যেমন "KF" মডেল) শিক্ষক বলেন তিনি জানেন না: এই জগৎ তাঁর আগ্রহের নয়। এখানে দরকারি কথা হলো **উৎপাদন লাইন** একটিই আর গতির পার্থক্য চিপ কেমন হলো তার ওপর একটি **বাছাই** ("ভালো হয়েছে? বেশি দামে বেচি")।

11 কাজ ও জমা

সতর্কতা

Von Neumann নিয়ে প্রতিবেদন: শিক্ষক **শুক্রবার** বা **আগামী সোমবারের** মধ্যে **Von Neumann কী করেছেন** তা নিয়ে একটি **ছোট প্রতিবেদন** আশা করেন ("জানি না, আগ্রহ হলে খুঁজে দেখো")। শিক্ষক একটি সিনেমা দেখাতে চেয়েছিলেন, কিন্তু তাঁর কাছে নেই। অন্য কাজ: "কাল সকালে এখন কিছু নেই" এবং "সব কোড আজ সন্ধ্যার পাঠ্যে পাবে": এই পাঠের অনুশীলনগুলো (অধিবর্ষ, ত্রিভুজ, অ্যালার্ম) **V02 — Esercizi Operatori Logici** পত্রের, যাতে আরও আছে (5 অ্যালার্মের তাপমাত্রা, 6 Luna Park-এর দোলনা, 7 জোড় ও ধনাত্মক সংখ্যা, 8 পাসওয়ার্ড ও PIN দিয়ে প্রবেশ)।

উদাহরণ

পত্রের 5–8 নম্বরের সঠিক লেখা (পর্দায় দেখা, ক্লাসে করা হয়নি)।

- **5. অ্যালার্মের তাপমাত্রা (Temperatura di Allarme).** C-তে একটি প্রোগ্রাম লেখো যা ঘরের তাপমাত্রা চায়। তাপমাত্রা 18-এর ছোট **অথবা** 26-এর বড় হলে প্রোগ্রাম ছাপাবে "Allarme temperatura", নাহলে "Temperatura OK"।
- **6. Luna Park-এর দোলনা (Giostra).** C-তে একটি প্রোগ্রাম লেখো যা সেন্টিমিটারে উচ্চতা ও বয়স চায়। উচ্চতা 140-এর বড় বা সমান **এবং** বয়স 10-এর বড় বা সমান হলে ছাপাবে "Puoi salire" (উঠতে পারো), নাহলে "Non puoi salire" (উঠতে পারো না)।
- **7. জোড় ও ধনাত্মক সংখ্যা (Numero Pari e Positivo).** C-তে একটি প্রোগ্রাম লেখো যা একটি পূর্ণসংখ্যা চায়। সংখ্যা 0-এর বড় **এবং** 2 দিয়ে ভাগের ভাগশেষ 0-এর সমান (`numero % 2 == 0`) হলে ছাপাবে "Positivo e pari" (ধনাত্মক ও জোড়), নাহলে "Condizione non verificata" (শর্ত পূরণ হয়নি)।
- **8. পাসওয়ার্ড ও PIN দিয়ে প্রবেশ (Accesso con Password e PIN).** C-তে একটি প্রোগ্রাম লেখো যা একটি নিরাপত্তা কোড ও একটি PIN (দুটি পূর্ণসংখ্যা) চায়। শুধু কোড 1234-এর সমান **এবং** PIN 5678-এর সমান হলে ছাপাবে "Accesso consentito" (প্রবেশ অনুমোদিত), নাহলে "Accesso negato" (প্রবেশ নিষেধ)।

12 এই পাঠ থেকে যা মনে রাখতে হবে

- **"Oppure"** = **OR** (`||`); **"e"** = **AND** (`&&`); **"non"** = **NOT** (`!`); **"diverso"** = `!=`।
- **অধিবর্ষ**: 400 দিয়ে বিভাজ্য, **অথবা** 4 দিয়ে বিভাজ্য **এবং** 100 দিয়ে **নয়**; বিভাজ্যতা যাচাই হয় **মডুলো** `%` শূন্যের সমান দিয়ে।
- **বৈধ ত্রিভুজ**: প্রতিটি বাহু অন্য দুটির **যোগফলের চেয়ে ছোট**, AND দিয়ে জোড়া তিনটি তুলনা।
- **শূন্য মিথ্যা**, অন্য প্রতিটি সংখ্যা **সত্য**; `!allarme` সত্য শুধু `allarme` **শূন্য** হলে।
- **ইন্ডেন্টেশন** বাধ্যতামূলক: বন্ধ বন্ধনী খোলার নিচে, ভেতরের অংশ **এক ট্যাব ডানে**; ইন্ডেন্ট না করা কোডের মান **শূন্য**।
- যৌগিক শর্তে সবসময় **গোল বন্ধনী**; একটি নির্দেশেও সবসময় **কার্লি বন্ধনী**।
- **মন্তব্য**: বহু-লাইন `/* ...*/`, এক লাইন `//`; **কম্পাইলার** উপেক্ষা করে; কোম্পানিতে **বাধ্যতামূলক**।
- **Firmware**: **ইলেকট্রনিক বোর্ড** চালানো প্রোগ্রাম; **মাইক্রোকন্ট্রোলারে** একটি **CPU** ও দুটি মেমোরি (প্রোগ্রাম ও ডেটা)।
- **Harvard**: দুটি আলাদা মেমোরি, বেশি দ্রুত (Arduino)। **Von Neumann**: একটি মেমোরি (পিসি)।

13 শব্দকোষ: ক্লাসে ব্যবহৃত ইতালীয় শব্দ

- **anno bisestile**: অধিবর্ষ
- **divisibile**: বিভাজ্য

- **lato / triangolo**: বাহু / ত্রিভুজ
- **allarme attivo / disattivato**: অ্যালার্ম চালু / বন্ধ
- **indentazione / tabulazione**: ইন্ডেন্টেশন / ট্যাব
- **commento**: মন্তব্য
- **scheda**: পত্র বা (ইলেকট্রনিক) বোর্ড
- **piedino**: চিপের পা
- **memoria programma / dati**: প্রোগ্রাম / ডেটা মেমোরি
- **relazione**: প্রতিবেদন